Trie (Prefix Tree) Templates

Quick Reference Table

#	Name	Description	Intuition	Time	Space	Key Implementation
208	Implement Trie	Implement `insert(word)`, `search(word)`, and `startsWith(prefix)`.	each word is a path of letters down the tree; shared prefixes share the same path, so searching is just walking that path.	O(k) per operation, k = word length	O(n·k) total	Standard
211	Design Add and Search Words Data Structure	Same as Trie but `search` supports `.` which matches any single letter.		O(k) insert, O(26^k) search worst case (all `.`)	O(n·k)	when character is `.`, recursively try every non-null child.
212	Word Search II	Given a board and a list of words, find all words that exist in the board (connected adjacent cells, no reuse).		O(M·N·4·3^(L-1)) where L = max word length	O(total chars in words)	build a Trie from the word list; during grid DFS, follow the Trie path to prune branches that can't form any word. This avoids re-scanning the grid for each word independently.
648	Replace Words	Replace every word in a sentence with its shortest root from a dictionary. If multiple roots are prefixes, use the shortest.		O(n·k) build + O(sentence) search	O(dict·k)	insert dictionary into Trie. For each word in sentence, traverse the Trie and return immediately when `isEnd` is hit — that's the shortest matching root.
1268	Search Suggestions System	For each prefix of `searchWord` (length 1, 2, ..., n), return up to 3 products from `products` that share that prefix, in lexicographic order.		O(n·k·log n) build + O(k) search	O(n·k)	sort products first, then insert into Trie. At each node on the insertion path, cache the word (up to 3) — since products are sorted, the first 3 cached are always lexicographically smallest. Lookup is then O(1) per prefix.
745	Prefix and Suffix Search	Design a class `WordFilter` with `f(pref, suff)` that returns the index of the word with the given prefix and suffix. If multiple words match, return the largest index.		O(W·L²) build, O(p+s) query	O(W·L²) where W=words.length, L=max word length	precompute all prefix-suffix pairs
336	Palindrome Pairs	Given a list of unique words, find all pairs of distinct indices `(i, j)` such that `words[i] + words[j]` is a palindrome.	insert every word **reversed** into a Trie, tagging each end node with its index. For a query word, walk the Trie char by char; whenever the remainder of the word is itself a palindrome and we sit on a complete reversed word, those two	O(n·k²) where n = words count, k = max word length	O(n·k)	words whose remaining suffix is a palindrome

#	Name	Description	Intuition	Time	Space	Key Implementation
			concatenate into a palindrome.			
676	Implement Magic Dictionary	Build a dictionary from a list of words. Implement `search(word)` that returns true if you can change **exactly one** character of `word` to make it match a dictionary word.	standard Trie insert; on search, DFS the Trie tracking how many characters have been changed so far. Allow exactly one mismatch.	O(k) insert, O(26^k) search worst case	O(n·k)	exactly one change required
720	Longest Word in Dictionary	Given a list of words, return the longest word that can be built one character at a time by other words in the list. If multiple, return the lexicographically smallest.	a word is buildable only if **every** prefix of it is also a complete word in the list. Insert all words, then DFS the Trie only through `isEnd` nodes — any reachable end node represents a fully buildable word.	O(n·k) build + O(n·k) DFS	O(n·k)	longest, ties broken lexicographically
1233	Remove Sub-Folders from the Filesystem	Given a list of folder paths, remove all sub-folders so only top-level folders remain. `"/a/b"` is a sub-folder of `"/a"`.	split each path into its `/` -separated components and build a Trie keyed by component (map-based, since components are arbitrary strings). Mark the node ending a folder. A folder is a sub-folder iff some ancestor node on its path is already an `isEnd` folder.	O(n·k) where n = folders, k = avg components	O(n·k)	map keyed by path component

When to Use Trie (vs HashMap/Array)

Signal	Use
Prefix queries / `startsWith` / autocomplete	**Trie** — shares prefixes, prefix lookup is O(k)
Wildcard match (`.` matches any char)	**Trie** — DFS branches over children
Prune a search space by shared prefixes (e.g. Word Search II)	**Trie** — dead branches cut early
Exact-key lookup only (no prefix logic)	**HashMap** — O(k) hash is simpler and faster, no node overhead

Canonical Template

```
// MENTAL MODEL: every node is a shared prefix; walking down spells a word, so common prefixes are stor
// WHEN: "prefix / startsWith / autocomplete", "wildcard match", "prune a search by shared prefixes"

// ── TRIE NODE ──
// Array vs Map TrieNode mental model:
//   array  = O(1) per char, fixed 26 letters, wastes space when sparse
//   HashMap = variable alphabet, smaller for sparse, hashing overhead
class TrieNode {
    TrieNode[] children = new TrieNode[26];   // array-based: O(1) access, O(26) per node
    boolean isEnd = false;
}
// Alternative: Map-based (for non-lowercase or variable alphabets)
class TrieNode {
    Map<Character, TrieNode> children = new HashMap<>();
    boolean isEnd = false;
}

// ── INSERT ──
void insert(TrieNode root, String word) {
    TrieNode node = root;
    for (char c : word.toCharArray()) {
        int idx = c - 'a';
        if (node.children[idx] == null) node.children[idx] = new TrieNode();
        node = node.children[idx];
    }
    node.isEnd = true;
}

// ── SEARCH (exact match) ──
boolean search(TrieNode root, String word) {
    TrieNode node = root;
    for (char c : word.toCharArray()) {
        int idx = c - 'a';
        if (node.children[idx] == null) return false;
        node = node.children[idx];
    }
    return node.isEnd;    // must end at a marked word boundary
}

// ── STARTS WITH (prefix match) ──
boolean startsWith(TrieNode root, String prefix) {
    TrieNode node = root;
    for (char c : prefix.toCharArray()) {
        int idx = c - 'a';
        if (node.children[idx] == null) return false;
        node = node.children[idx];
    }
    return true;          // ← difference from search: don't check node.isEnd
}
```

Variations

```
// VARIATION 1: DFS for wildcard '.' (Add and Search Words)
// When char is '.', try all 26 children recursively instead of following one path.
boolean dfs(TrieNode node, String word, int i) {
    if (i == word.length()) return node.isEnd;
    char c = word.charAt(i);
    if (c == '.') {
        for (TrieNode child : node.children)            // ← VARIATION: try all children
            if (child != null && dfs(child, word, i+1)) return true;
        return false;
    }
    int idx = c - 'a';
    if (node.children[idx] == null) return false;
    return dfs(node.children[idx], word, i + 1);
}

// VARIATION 2: early-exit on isEnd (Replace Words — find shortest root)
String findRoot(TrieNode root, String word) {
    TrieNode node = root;
    for (int i = 0; i < word.length(); i++) {
        int idx = word.charAt(i) - 'a';
        if (node.children[idx] == null) return word;    // no root found → return original
        node = node.children[idx];
        if (node.isEnd) return word.substring(0, i+1);  // ← VARIATION: return at first root hit
    }
    return word;
}

// VARIATION 3: store words at each node (Search Suggestions — top-3 per prefix)
// During insert, each node on the path caches the word (up to 3).
// Products must be inserted in sorted order → first 3 are lexicographically smallest.
void insert(TrieNode root, String word) {
    TrieNode node = root;
    for (char c : word.toCharArray()) {
        int idx = c - 'a';
        if (node.children[idx] == null) node.children[idx] = new TrieNode();
        node = node.children[idx];
        if (node.words.size() < 3) node.words.add(word);  // ← VARIATION: cache at every node
    }
    node.isEnd = true;
}

// VARIATION 4: Trie + grid DFS pruning (Word Search II)
// Build Trie from word list. DFS the grid; at each cell, check if current path
// follows a Trie path. Prune entire branch if no Trie node exists.
void dfs(char[][] board, int i, int j, TrieNode node, StringBuilder path, Set<String> result) {
    if (i < 0 || i >= board.length || j < 0 || j >= board[0].length) return;
    char c = board[i][j];
    if (c == '#' || node.children[c - 'a'] == null) return;  // ← VARIATION: Trie prune
    node = node.children[c - 'a'];
    path.append(c);
    if (node.isEnd) result.add(path.toString());
    board[i][j] = '#';
    dfs(board, i+1, j, node, path, result);
    dfs(board, i-1, j, node, path, result);
    dfs(board, i, j+1, node, path, result);
    dfs(board, i, j-1, node, path, result);
    board[i][j] = c;
    path.deleteCharAt(path.length() - 1);
}
```

search vs startsWith — One Line Difference

```
search:      return node.isEnd;   // must land on a word boundary
startsWith:  return true;         // any node reached = valid prefix
```

Part 1 — Standard Trie

#208 Implement Trie

Description: Implement `insert(word)`, `search(word)`, and `startsWith(prefix)`.

Intuition: each word is a path of letters down the tree; shared prefixes share the same path, so searching is just walking that path.

```java
class Trie {
    private TrieNode root = new TrieNode();

    static class TrieNode {
        TrieNode[] children = new TrieNode[26];
        boolean isEnd = false;
    }

    public void insert(String word) {
        TrieNode node = root;
        for (char c : word.toCharArray()) {
            int idx = c - 'a';
            if (node.children[idx] == null) node.children[idx] = new TrieNode();
            node = node.children[idx];
        }
        node.isEnd = true;
    }

    public boolean search(String word) {
        TrieNode node = root;
        for (char c : word.toCharArray()) {
            int idx = c - 'a';
            if (node.children[idx] == null) return false;
            node = node.children[idx];
        }
        return node.isEnd;          // ← must be marked as a complete word
    }

    public boolean startsWith(String prefix) {
        TrieNode node = root;
        for (char c : prefix.toCharArray()) {
            int idx = c - 'a';
            if (node.children[idx] == null) return false;
            node = node.children[idx];
        }
        return true;                // ← any reachable node = valid prefix
    }
}
```

Time O(k) per operation, k = word length | **Space** O(n·k) total

Part 2 — Wildcard DFS

#211 Design Add and Search Words Data Structure

Description: Same as Trie but `search` supports `.` which matches any single letter.
Variation: when character is `.`, recursively try every non-null child.

```
class WordDictionary {
    private TrieNode root = new TrieNode();

    static class TrieNode {
        TrieNode[] children = new TrieNode[26];
        boolean isEnd = false;
    }

    public void addWord(String word) {
        TrieNode node = root;
        for (char c : word.toCharArray()) {
            int idx = c - 'a';
            if (node.children[idx] == null) node.children[idx] = new TrieNode();
            node = node.children[idx];
        }
        node.isEnd = true;
    }

    public boolean search(String word) {
        return dfs(root, word, 0);
    }

    private boolean dfs(TrieNode node, String word, int i) {
        if (i == word.length()) return node.isEnd;
        char c = word.charAt(i);
        if (c == '.') {
            for (TrieNode child : node.children)             // ← VARIATION: try all 26 children
                if (child != null && dfs(child, word, i+1)) return true;
            return false;
        }
        int idx = c - 'a';
        if (node.children[idx] == null) return false;
        return dfs(node.children[idx], word, i + 1);
    }
}
```

Time O(k) insert, O(26^k) search worst case (all `.`) | **Space** O(n·k)

Part 3 — Trie + Grid DFS

#212 Word Search II

Description: Given a board and a list of words, find all words that exist in the board (connected adjacent cells, no reuse).
Variation: build a Trie from the word list; during grid DFS, follow the Trie path to prune branches that can't form any word. This avoids re-scanning the grid for each word independently.

```java
class Solution {
    int[] dr = {1, -1, 0,  0};
    int[] dc = {0,  0, 1, -1};

    public List<String> findWords(char[][] board, String[] words) {
        TrieNode root = new TrieNode();
        for (String word : words) insert(root, word);         // ← VARIATION: build Trie first

        Set<String> result = new HashSet<>();
        for (int i = 0; i < board.length; i++)
            for (int j = 0; j < board[0].length; j++)
                dfs(board, i, j, root, new StringBuilder(), result);
        return new ArrayList<>(result);
    }

    private void insert(TrieNode root, String word) {
        TrieNode node = root;
        for (char c : word.toCharArray()) {
            int idx = c - 'a';
            if (node.children[idx] == null) node.children[idx] = new TrieNode();
            node = node.children[idx];
        }
        node.isEnd = true;
    }

    private void dfs(char[][] board, int i, int j, TrieNode node, StringBuilder path, Set<String> resul
        if (i < 0 || i >= board.length || j < 0 || j >= board[0].length) return;
        char c = board[i][j];
        if (c == '#') return;                                  // already visited
        int idx = c - 'a';
        if (node.children[idx] == null) return;               // ← VARIATION: Trie prune — no path here
        node = node.children[idx];
        path.append(c);
        if (node.isEnd) result.add(path.toString());          // found a word
        board[i][j] = '#';                                     // mark visited
        for (int d = 0; d < 4; d++)
            dfs(board, i + dr[d], j + dc[d], node, path, result);
        board[i][j] = c;                                       // restore
        path.deleteCharAt(path.length() - 1);
    }

    static class TrieNode {
        TrieNode[] children = new TrieNode[26];
        boolean isEnd = false;
    }
}
```

Time O(M·N·4·3^(L-1)) where L = max word length | **Space** O(total chars in words)

Part 4 — Early-Exit Search

#648 Replace Words

Description: Replace every word in a sentence with its shortest root from a dictionary. If multiple roots are prefixes, use the shortest.
Variation: insert dictionary into Trie. For each word in sentence, traverse the Trie and return immediately when `isEnd` is hit — that's the shortest matching root.

```java
class Solution {
    public String replaceWords(List<String> dictionary, String sentence) {
        TrieNode root = new TrieNode();
        for (String root_ : dictionary) insert(root, root_);

        StringBuilder result = new StringBuilder();
        for (String word : sentence.split(" ")) {
            if (result.length() > 0) result.append(" ");
            result.append(findRoot(root, word));
        }
        return result.toString();
    }

    private String findRoot(TrieNode root, String word) {
        TrieNode node = root;
        for (int i = 0; i < word.length(); i++) {
            int idx = word.charAt(i) - 'a';
            if (node.children[idx] == null) return word;       // no root prefix found
            node = node.children[idx];
            if (node.isEnd) return word.substring(0, i + 1);   // ← VARIATION: return at first root
        }
        return word;
    }

    private void insert(TrieNode root, String word) {
        TrieNode node = root;
        for (char c : word.toCharArray()) {
            int idx = c - 'a';
            if (node.children[idx] == null) node.children[idx] = new TrieNode();
            node = node.children[idx];
        }
        node.isEnd = true;
    }

    static class TrieNode {
        TrieNode[] children = new TrieNode[26];
        boolean isEnd = false;
    }
}
```

Time O(n·k) build + O(sentence) search | **Space** O(dict·k)

Part 5 — Words Cached at Node

#1268 Search Suggestions System

Description: For each prefix of `searchWord` (length 1, 2, ..., n), return up to 3 products from `products` that share that prefix, in lexicographic order.
Variation: sort products first, then insert into Trie. At each node on the insertion path, cache the word (up to 3) — since products are sorted, the first 3 cached are always lexicographically smallest. Lookup is then O(1) per prefix.

```java
class Solution {
    static class TrieNode {
        TrieNode[] children = new TrieNode[26];
        List<String> words = new ArrayList<>();    // ← VARIATION: cache words at each node
    }

    public List<List<String>> suggestedProducts(String[] products, String searchWord) {
        Arrays.sort(products);                     // ← VARIATION: sort first so cache is ordered
        TrieNode root = new TrieNode();
        for (String p : products) {
            TrieNode node = root;
            for (char c : p.toCharArray()) {
                int idx = c - 'a';
                if (node.children[idx] == null) node.children[idx] = new TrieNode();
                node = node.children[idx];
                if (node.words.size() < 3) node.words.add(p); // ← VARIATION: at most 3 per node
            }
        }

        List<List<String>> result = new ArrayList<>();
        TrieNode node = root;
        boolean dead = false;
        for (char c : searchWord.toCharArray()) {
            if (!dead && node.children[c - 'a'] != null) {
                node = node.children[c - 'a'];
                result.add(new ArrayList<>(node.words)); // ← O(1) lookup: words already cached
            } else {
                dead = true;
                result.add(new ArrayList<>());
            }
        }
        return result;
    }
}
```

Time O(n·k·log n) build + O(k) search | **Space** O(n·k)

Trie vs HashMap vs Sorting — When to Use Trie

Operation	HashMap	Sorting	Trie
Exact key lookup	O(k) ✓	O(log n · k)	O(k)
Prefix check	O(k) per key	O(log n)	O(k) ✓
All words with prefix	O(n · k)	O(log n + result)	O(k + result) ✓
Wildcard match	Not supported	Not supported	O(26^k) with DFS ✓
Space	O(n · k)	O(n · k)	O(n · k) alphabet-dependent

Use Trie when: prefix queries, wildcard search, or pruning a search space by shared prefixes (Word Search II).

Insert → Search — Structural Parallel

```
Both traverse the same path. Insert creates nodes; search reads them.
The only decision point after traversal:

  search:      return node.isEnd         ← complete word?
  startsWith:  return true               ← any prefix is fine
  findRoot:    return early if isEnd     ← want the shallowest word boundary
  cache words: node.words.add(word)      ← accumulate at every node during insert
```

Part 6 — Prefix + Suffix Lookup

#745 Prefix and Suffix Search

Description: Design a class `WordFilter` with `f(pref, suff)` that returns the index of the word with the given prefix and suffix. If multiple words match, return the largest index.

Algorithm: Precompute all `(prefix + "#" + suffix) → index` pairs for every word during construction. Query is just a HashMap lookup. Since we insert in order, later indices naturally overwrite earlier ones.

```java
class WordFilter {
    private final Map<String, Integer> map = new HashMap<>();   // ← VARIATION: precompute all prefix-s

    public WordFilter(String[] words) {
        for (int i = 0; i < words.length; i++) {
            String w = words[i];
            int n = w.length();
            for (int p = 0; p <= n; p++) {
                String prefix = w.substring(0, p);
                for (int s = 0; s <= n; s++) {
                    String suffix = w.substring(n - s);
                    map.put(prefix + "#" + suffix, i);   // ← VARIATION: later index overwrites earlier
                }
            }
        }
    }

    public int f(String pref, String suff) {
        return map.getOrDefault(pref + "#" + suff, -1);
    }
}
```

Time O(W·L²) build, O(p+s) query | **Space** O(W·L²) where W=words.length, L=max word length

---

Additional Reference Problems

#336 Palindrome Pairs

Description: Given a list of unique words, find all pairs of distinct indices `(i, j)` such that `words[i] + words[j]` is a palindrome.

Intuition: insert every word **reversed** into a Trie, tagging each end node with its index. For a query word, walk the Trie char by char; whenever the remainder of the word is itself a palindrome and we sit on a complete reversed word, those two concatenate into a palindrome.

Algorithm: store at each `isEnd` node the word's index, and at every node a list of indices whose remaining (un-traversed) suffix is a palindrome. While matching `words[i]` against the reversed-word Trie, collect index matches that pair into palindromes.

```java
class Solution {
    static class TrieNode {
        TrieNode[] children = new TrieNode[26];
        int index = -1;                 // index of word ending here (reversed), -1 if none
        List<Integer> palindromeBelow = new ArrayList<>();  // ← VARIATION: words whose remaining suffi
    }

    public List<List<Integer>> palindromePairs(String[] words) {
        TrieNode root = new TrieNode();
        for (int i = 0; i < words.length; i++) {
            insert(root, words[i], i);
        }

        List<List<Integer>> result = new ArrayList<>();
        for (int i = 0; i < words.length; i++) {
            search(root, words[i], i, result);
        }
        return result;
    }

    private void insert(TrieNode root, String word, int index) {
        TrieNode node = root;
        int n = word.length();
        for (int j = n - 1; j >= 0; j--) {                  // ← VARIATION: insert reversed
            if (isPalindrome(word, 0, j)) {
                node.palindromeBelow.add(index);            // prefix [0..j] is a palindrome
            }
            int idx = word.charAt(j) - 'a';
            if (node.children[idx] == null) {
                node.children[idx] = new TrieNode();
            }
            node = node.children[idx];
        }
        node.palindromeBelow.add(index);                    // empty remainder is a palindrome
        node.index = index;
    }

    private void search(TrieNode root, String word, int i, List<List<Integer>> result) {
        TrieNode node = root;
        int n = word.length();
        for (int k = 0; k < n; k++) {
            // case: matched a full reversed word AND the rest of this word is a palindrome
            if (node.index != -1 && node.index != i && isPalindrome(word, k, n - 1)) {
                result.add(Arrays.asList(i, node.index));
            }
            int idx = word.charAt(k) - 'a';
            if (node.children[idx] == null) {
                return;
            }
            node = node.children[idx];
        }
        // consumed all of word: pair with any reversed word whose extra suffix is a palindrome
        for (int j : node.palindromeBelow) {
            if (j != i) {
                result.add(Arrays.asList(i, j));
            }
        }
    }

    private boolean isPalindrome(String word, int left, int right) {
        while (left < right) {
            if (word.charAt(left) != word.charAt(right)) {
                return false;
```

```
            }
            left++;
            right--;
        }
        return true;
    }
}
```

Time O(n·k²) where n = words count, k = max word length | **Space** O(n·k)

#676 Implement Magic Dictionary

Description: Build a dictionary from a list of words. Implement `search(word)` that returns true if you can change **exactly one** character of `word` to make it match a dictionary word.

Intuition: standard Trie insert; on search, DFS the Trie tracking how many characters have been changed so far. Allow exactly one mismatch.

Algorithm: at each position try the matching child with `changed` unchanged, and (if budget remains) every other child with `changed = 1`. Succeed only when the whole word is consumed, the node is a word end, and exactly one change was used.

```java
class MagicDictionary {
    private TrieNode root = new TrieNode();

    static class TrieNode {
        TrieNode[] children = new TrieNode[26];
        boolean isEnd = false;
    }

    public void buildDict(String[] dictionary) {
        for (String word : dictionary) {
            insert(word);
        }
    }

    private void insert(String word) {
        TrieNode node = root;
        for (char c : word.toCharArray()) {
            int idx = c - 'a';
            if (node.children[idx] == null) {
                node.children[idx] = new TrieNode();
            }
            node = node.children[idx];
        }
        node.isEnd = true;
    }

    public boolean search(String word) {
        return dfs(root, word, 0, 0);
    }

    private boolean dfs(TrieNode node, String word, int i, int changed) {
        if (i == word.length()) {
            return node.isEnd && changed == 1;              // ← VARIATION: exactly one change required
        }
        int idx = word.charAt(i) - 'a';
        for (int c = 0; c < 26; c++) {
            if (node.children[c] == null) {
                continue;
            }
            int nextChanged = (c == idx) ? changed : changed + 1;
            if (nextChanged > 1) {
                continue;                                   // ← VARIATION: prune after a second change
            }
            if (dfs(node.children[c], word, i + 1, nextChanged)) {
                return true;
            }
        }
        return false;
    }
}
```

Time O(k) insert, O(26^k) search worst case | **Space** O(n·k)

#720 Longest Word in Dictionary

Description: Given a list of words, return the longest word that can be built one character at a time by other words in the list. If multiple, return the lexicographically smallest.

Intuition: a word is buildable only if **every** prefix of it is also a complete word in the list. Insert all words, then DFS the Trie only through `isEnd` nodes — any reachable end node represents a fully buildable word.

Algorithm: DFS from root; descend only into children that are themselves word ends. Track the best (longest, then lexicographically smallest) reachable word.

```java
class Solution {
    static class TrieNode {
        TrieNode[] children = new TrieNode[26];
        boolean isEnd = false;
    }

    private String best = "";

    public String longestWord(String[] words) {
        TrieNode root = new TrieNode();
        for (String word : words) {
            insert(root, word);
        }
        dfs(root, new StringBuilder());
        return best;
    }

    private void insert(TrieNode root, String word) {
        TrieNode node = root;
        for (char c : word.toCharArray()) {
            int idx = c - 'a';
            if (node.children[idx] == null) {
                node.children[idx] = new TrieNode();
            }
            node = node.children[idx];
        }
        node.isEnd = true;
    }

    private void dfs(TrieNode node, StringBuilder builder) {
        String word = builder.toString();
        if (word.length() > best.length()
                || (word.length() == best.length() && word.compareTo(best) < 0)) {
            best = word;                                    // ← VARIATION: longest, ties broken lexico
        }
        for (int i = 0; i < 26; i++) {
            TrieNode child = node.children[i];
            if (child != null && child.isEnd) {             // ← VARIATION: only descend through word e
                builder.append((char) ('a' + i));
                dfs(child, builder);
                builder.deleteCharAt(builder.length() - 1);
            }
        }
    }
}
```

Time O(n·k) build + O(n·k) DFS | **Space** O(n·k)

#1233 Remove Sub-Folders from the Filesystem

Description: Given a list of folder paths, remove all sub-folders so only top-level folders remain. `"/a/b"` is a sub-folder of `"/a"`.

Intuition: split each path into its `/`-separated components and build a Trie keyed by component (map-based, since components are arbitrary strings). Mark the node ending a folder. A folder is a sub-folder iff some ancestor node on its path is already an `isEnd` folder.

Algorithm: insert all folders; then for each folder walk its components and keep it only if no ancestor along the way is marked as a folder end.

```java
class Solution {
    static class TrieNode {
        Map<String, TrieNode> children = new HashMap<>();   // ← VARIATION: map keyed by path component
        boolean isEnd = false;
    }

    public List<String> removeSubfolders(String[] folder) {
        TrieNode root = new TrieNode();
        for (String path : folder) {
            insert(root, path);
        }

        List<String> result = new ArrayList<>();
        for (String path : folder) {
            if (isTopLevel(root, path)) {
                result.add(path);
            }
        }
        return result;
    }

    private void insert(TrieNode root, String path) {
        TrieNode node = root;
        for (String part : path.split("/")) {
            if (part.isEmpty()) {                           // skip the leading empty token from leadin
                continue;
            }
            node.children.putIfAbsent(part, new TrieNode());
            node = node.children.get(part);
        }
        node.isEnd = true;
    }

    private boolean isTopLevel(TrieNode root, String path) {
        TrieNode node = root;
        String[] parts = path.split("/");
        for (int i = 0; i < parts.length; i++) {
            if (parts[i].isEmpty()) {
                continue;
            }
            node = node.children.get(parts[i]);
            if (node.isEnd && !isLast(parts, i)) {          // ← VARIATION: ancestor folder end → sub-f
                return false;
            }
        }
        return true;
    }

    private boolean isLast(String[] parts, int i) {
        for (int j = i + 1; j < parts.length; j++) {
            if (!parts[j].isEmpty()) {
                return false;
            }
        }
        return true;
    }
}
```

Time O(n·k) where n = folders, k = avg components | **Space** O(n·k)